SnapEvents Demo Plan: 100% GitHub Pages Architecture (`github.io`)

> [!IMPORTANT]
> **Will a GitHub Pages Demo Work? Yes — 100%!** Because `https://api.hel.fi/linkedevents/v1/` and `https://tiles.openfreemap.org` both serve `Access-Control-Allow-Origin: *` over HTTPS, and because GitHub Pages serves over `https://<username>.github.io/<repo-name>/` with native HTTPS (required for browser `navigator.geolocation` and WebGL), **every single feature** of the 6-city Snap Map will work on GitHub Pages with **zero backend servers and zero impact on your Cloudflare account**.

Part 1: Why GitHub Pages Works Out-of-the-Box (Feasibility Matrix)

Feature / Requirement	Works on GitHub Pages?	Why It Works & Technical Mechanism
Live Events for Helsinki, Espoo & Vantaa	✅ Yes	Browser on `https://<user>.github.io` fetches `https://api.hel.fi/linkedevents/v1/event/?division=helsinki,espoo,vantaa&include=location,keywords` directly via CORS (`Access-Control-Allow-Origin: *`).
Events for Turku, Tampere & Oulu	✅ Yes	Served as static JSON-LD files (`${BASE_URL}feeds/{turku,tampere,oulu}.json`) bundled into the GitHub Pages deployment artifact by GitHub Actions.
MapLibre GL JS + Custom Snap Styles	✅ Yes	Runs 100% client-side in WebGL; vector tiles stream directly from `https://tiles.openfreemap.org` (no API key or domain whitelist needed).
Browser "Near Me" Geolocation	✅ Yes	GitHub Pages enforces HTTPS on `*.github.io` , satisfying the browser Secure Context requirement for `navigator.geolocation` .
Shared Deep Links to Specific Events	✅ Yes	Using query parameters (`?city=helsinki&event=lippupiste:serie-4221822`) instead of path routing (`/event/...`) avoids GitHub Pages 404 errors on refresh.
Cloudflare Account Quota Impact	✅ **0.0%**	Hosted entirely on GitHub's Fastly CDN infrastructure—completely isolated from your Cloudflare account.

Part 2: Four GitHub Pages–Specific Gotchas & How We Solve Them

Deploying a rich interactive SPA to `https://<username>.github.io/<repo-name>/` has four platform-specific quirks that do not exist on root-domain hosts. Our demo architecture solves all four up front:

1. Subpath Base URL (`/<repo-name>/`)

Unless you attach a custom domain, GitHub Pages serves project repositories from a subpath (`https://<username>.github.io/snapevents-demo/`) rather than `/` .

- **Problem:** Hardcoded absolute paths like `fetch('/feeds/turku.json')` break because they request `https://<username>.github.io/feeds/turku.json` (404).
- **Solution:**
  - Set `base: process.env.VITE_BASE_PATH || '/snapevents-demo/'` in `vite.config.ts` .

- Always resolve static feeds and assets with Vite's built-in `import.meta.env.BASE_URL`:

```ts
const feedUrl = `${import.meta.env.BASE_URL}feeds/${cityId}.json`;
```

2. SPA Routing & Avoiding GitHub Pages `404.html` Traps

GitHub Pages does not support server-side URL rewrites (`_redirects` or `try_files $uri /index.html`).

- **Problem:** If a user opens a path-based URL like `https://<user>.github.io/snapevents-demo/helsinki/event/123`, GitHub Pages looks for a physical file at `/helsinki/event/123/index.html` and returns a 404 error.
- **Solution — Query-Parameter Deep Linking:**
  - Keep the application at the single root entry point (`https://<user>.github.io/snapevents-demo/`) and store all shareable state in **URL Search Params**:

```text
https://<user>.github.io/snapevents-demo/?
city=helsinki&time=tonight&topic=music&event=lippupiste:serie-4221822
```

  - Every shared link hits `index.html` directly with a `200 OK` status, restores the exact camera position/city/filter state, and automatically opens the 9:16 Snap Story Player if `?event=...` is present.

3. Updating Static Feeds Without Polluting Git Commit History

If a scheduled GitHub Action commits updated `turku.json`, `tampere.json`, and `oulu.json` back into the `main` branch every 6 hours, your repository commit log fills up with hundreds of automated bot commits.

- **Solution — Artifact-Based Pages Deployment (`actions/deploy-pages`):**
  - Instead of committing generated feeds to `main`, our `.github/workflows/deploy-demo.yml` workflow runs on `push`, `workflow_dispatch`, and `schedule` (`cron: '0 */12 * * *'`).
  - Inside the ephemeral GitHub Actions runner, it runs `npm run sync-feeds` (populating `public/feeds/{turku,tampere,oulu}.json`), builds `dist/`, and publishes `dist/` directly to the **GitHub Pages environment** via `actions/upload-pages-artifact` + `actions/deploy-pages`.
  - Your git history stays 100% clean, while the live demo updates automatically!

4. GitHub Pages Bandwidth & Size Limits

- **GitHub Pages Soft Limits:** 1 GB site size, 100 GB/month bandwidth, 10 builds/hour.
- **Why We Stay Under 1% of GitHub's Limits:**
  - Map vector tiles stream directly from `tiles.openfreemap.org` to the user's browser.
  - Event images (`event.images[0].url`) stream directly from source CDNs (`api.hel.fi` , `lippu.fi` , `espoo.fi`) to the user's browser.
  - Helsinki, Espoo, and Vantaa JSON-LD payloads stream directly from `api.hel.fi` to the user's browser and are cached in `IndexedDB` for 15 minutes.
  - GitHub Pages only serves the `< 500 KB` gzipped JS/CSS bundle and the three compressed static feed JSON files!

Part 3: System Architecture on GitHub Pages

Part 4: Data Licensing & GitHub Pages Terms of Service

All **Linked Events API licensing rules** remain identical to the main plan, with one additional hosting policy detail specific to GitHub Pages:

1. Linked Events Data & Image Compliance (Summary)

- **Event Metadata, Places & Keywords (`CC BY 4.0`):** Free to use, remix, and display on your public GitHub Pages demo as long as you credit the source ("*Event data: City of Helsinki / Espoo / Vantaa / 6Aika Linked Events API*") and link to CC BY 4.0.
- **Event Images (`event_only` vs. `cc_by`):**
  - Images marked `"license": "event_only"` **may** be displayed inside the map marker bubble and 9:16 Story Card for that specific event.
  - They **must not** be reused out of context (e.g., as fallback category backgrounds for other events or generic repo social preview banners).
  - Every Story Card displaying an image **must** show the photographer and source attribution pill (`📷 {photographer_name} • {provider || data_source}`).

2. GitHub Pages Acceptable Use Policy (Demo vs. Commercial Production)

> NOTE
>
> While the **Linked Events `CC BY 4.0` license** permits commercial use, **GitHub Pages' Terms of Service** prohibit using `github.io` to run a commercial business or paid SaaS product.
>
> - **For a Demo, Portfolio Showcase, Civic Tech Prototype, or Open-Source App:** GitHub Pages is **100% permitted and ideal**.
> - **Zero Lock-In Migration Path:** Because our build output (`dist/`) is a standard static Vite bundle, you can run the demo on GitHub Pages today and deploy the exact same repository to Cloudflare Pages later just by setting `VITE_BASE_PATH=/` .

Part 5: 6-City Data Strategy in the GitHub Pages Demo

City	Coordinates (`[lon, lat]`)	Demo Data Mechanism	Source & Fallback Behavior
1. Helsinki	`[24.9384, 60.1699]`	**Live Browser Fetch**	`api.hel.fi/linkedevents/v1/event/?division=helsinki&include=location,keywords` (plus bundled fallback sample if offline).
2. Espoo	`[24.6559, 60.2055]`	**Live Browser Fetch**	`api.hel.fi/linkedevents/v1/event/?division=espoo&include=location,keywords`
3. Vantaa	`[25.0409, 60.2934]`	**Live Browser Fetch**	`api.hel.fi/linkedevents/v1/event/?division=vantaa&include=location,keywords`
4. Turku	`[22.2666, 60.4518]`	**Static JSON-LD Feed**	`${BASE_URL}feeds/turku.json` — Pre-populated with rich, realistic `LinkedEvent` -compliant events at real Turku venues (*Logomo*, *Turun kaupunginteatteri*, *Turun pääkirjasto*, *Kupittaanpuisto*) and refreshed via CI.
5. Tampere	`[23.7610, 61.4978]`	**Static JSON-LD Feed**	`${BASE_URL}feeds/tampere.json` — Pre-populated with `LinkedEvent` -compliant events at real Tampere venues (*Tampere-talo*, *Nokia Arena*, *Pääkirjasto Metso*, *keskustori*) and refreshed via CI.
6. Oulu	`[25.4651, 65.0121]`	**Static JSON-LD Feed**	`${BASE_URL}feeds/oulu.json` — Pre-populated with `LinkedEvent` -compliant events at real Oulu venues (*Oulun teatteri*, *Kulttuuritalo Valve*, *Ouluhalli*, *Nallikari*) and refreshed via CI.

> TIP

> **Dynamic Date Shifting for Demo Reliability:** In the demo's static feeds for Turku, Tampere, and Oulu (`turku.json`, `tampere.json`, `oulu.json`), the client normalizer can optionally anchor relative demo event timestamps to the current day (`today` / `now` / `weekend`) if a static feed hasn't been synced recently—ensuring that clicking **Turku**, **Tampere**, or **Oulu** in the demo *always* lights up the thermal heatmap and pulsing `"LIVE"` Story Rings!

Part 6: Step-by-Step Implementation Plan for the GitHub Pages Demo

Phase 1: Repository & Vite Configuration for `github.io`

1. Scaffold **Vite + React + TypeScript + Tailwind CSS**.
2. Configure `vite.config.ts` with dynamic base path support:

```
ts
export default defineConfig({
  plugins: [react(), tailwindcss()],
  base: process.env.VITE_BASE_PATH || './',
});
```

(Using `base: './'` makes the built `dist/` folder portable so it works on any GitHub Pages repo name `https://<user>.github.io/<any-repo-name>/` AND local preview without changing config!)

Phase 2: 6Aika Data Client, Static Feeds & License Guard

1. Create `src/types/linkedEvents.ts` and `src/config/cities.ts` for all 6 cities (Helsinki, Espoo, Vantaa, Tampere, Turku, Oulu).
2. Create `public/feeds/{turku,tampere,oulu}.json` in strict 6Aika `LinkedEvent` JSON-LD format so the demo works out-of-the-box for all 6 cities immediately after `git push`.
3. Build `src/services/linkedEventsClient.ts`:
    - Fetches Helsinki, Espoo, and Vantaa live from `api.hel.fi/linkedevents/v1/event/` with `IndexedDB` caching (15-min TTL).
    - Fetches Turku, Tampere, and Oulu from `${import.meta.env.BASE_URL}feeds/${city}.json` (with dynamic timestamp anchoring so `"🔥 Live Now"` and `"🌙 Tonight"` filters always showcase active events across all 6 cities).
4. Build `src/utils/licenseGuard.ts` to format mandatory `event_only` and `CC BY 4.0` photographer/source credits.

Phase 3: Snap Map WebGL Engine (`MapLibre GL JS` + `OpenFreeMap`)

1. Build `src/components/Map/SnapMap.tsx` using `maplibre-gl` and `https://tiles.openfreemap.org/styles/dark`.
2. Add the runtime **Map Style Customizer** (🔮 *Snap Midnight*, ⚡ *Nordic Neon*, 🌞 *Snap Daylight*, plus 3D extruded buildings at `zoom >= 14.5`).
3. Add the **GPU Thermal Heatmap Layer** (weighted by live status and venue event count) and **Supercluster + Venue Story Bubble Markers** (pulsing neon rings, event count badges, category emojis).

Phase 4: 9:16 Story Player, URL Deep-Linking & GitHub Actions Workflow

1. Build the **9:16 Vertical Snap Story Viewer Modal** with segmented progress bars, `DOMPurify` HTML description drawer, mandatory image credit badge, Digitransit/Reittiopas transit directions link, and query-param deep-link sharing (`?city=...&event=...`).
2. Add `.github/workflows/deploy-demo.yml` using `actions/checkout@v4`, `actions/setup-node@v4`, `actions/upload-pages-artifact@v3`, and `actions/deploy-pages@v4` for automatic one-push deployment to GitHub Pages.